

3714. Longest Balanced Substring II

Solved 

Medium

 Topics

 Companies

 Hint

You are given a string `s` consisting only of the characters `'a'`, `'b'`, and `'c'`.

A **substring** of `s` is called **balanced** if all **distinct** characters in the **substring** appear the **same** number of times.

Return the **length of the longest balanced substring** of `s`.

Example 1:

Input: `s = "abbac"`

Output: 4

Explanation:

The longest balanced substring is `"abba"` because both distinct characters `'a'` and `'b'` each appear exactly 2 times.

Example 2:

Input: `s = "aabcc"`

Output: 3

Explanation:

The longest balanced substring is `"abc"` because all distinct characters `'a'`, `'b'` and `'c'` each appear exactly 1 time.

Example 3:

Input: `s = "aba"`

Output: 2

Explanation:

One of the longest balanced substrings is `"ab"` because both distinct characters `'a'` and `'b'` each appear exactly 1 time. Another longest balanced substring is `"ba"`.

Constraints:

- `1 <= s.length <= 105`
- `s` contains only the characters `'a'`, `'b'`, and `'c'`.

Python:

```
class Solution:
    def mono(self, s: str) -> int:
        if not s:
            return 0
        cnt = 1
        ans = 1
        for i in range(1, len(s)):
            if s[i] == s[i - 1]:
                cnt += 1
            else:
                cnt = 1
            ans = max(ans, cnt)
```

```
return ans
```

```
def duo(self, s: str, c1: str, c2: str) -> int:
```

```
    pos: Dict[int, int] = {0: -1}
```

```
    ans = 0
```

```
    delta = 0
```

```
    for i, ch in enumerate(s):
```

```
        if ch != c1 and ch != c2:
```

```
            pos.clear()
```

```
            pos[0] = i
```

```
            delta = 0
```

```
            continue
```

```
        if ch == c1:
```

```
            delta += 1
```

```
        else:
```

```
            delta -= 1
```

```
        if delta in pos:
```

```
            ans = max(ans, i - pos[delta])
```

```
        else:
```

```
            pos[delta] = i
```

```
return ans
```

```
def trio(self, s: str) -> int:
```

```
    cnt0 = cnt1 = cnt2 = 0
```

```
    pos: Dict[Tuple[int, int], int] = {(0, 0): -1}
```

```
    ans = 0
```

```
    for i, ch in enumerate(s):
```

```
        if ch == 'a':
```

```
            cnt0 += 1
```

```
        elif ch == 'b':
```

```
            cnt1 += 1
```

```
        else:
```

```
            cnt2 += 1
```

```
    key = (cnt1 - cnt0, cnt2 - cnt0)
```

```
    if key in pos:
```

```
        ans = max(ans, i - pos[key])
```

```
    else:
```

```
        pos[key] = i
```

```
return ans
```

```
def longestBalanced(self, s: str) -> int:  
    return max(  
        self.mono(s),  
        self.duo(s, 'a', 'b'),  
        self.duo(s, 'a', 'c'),  
        self.duo(s, 'b', 'c'),  
        self.trio(s),  
    )
```

JavaScript:

```
function longestBalanced(s) {  
    let res = 0  
  
    // Case 1  
    for (let i = 0; i < s.length; i++) {  
        let j = i  
        while (j+1 < s.length && s[j+1] === s[i]) {  
            j++  
        }  
        res = Math.max(res, j-i+1)  
        i = j  
    }  
  
    // Case 2  
    function checkTwo(a, b) {  
        let diff = 0  
        let map = { 0: -1 }  
        for (let i = 0; i < s.length; i++) {  
            if (s[i] === a) {  
                diff++  
            } else if (s[i] === b) {  
                diff--  
            } else {  
                map = { 0: i }  
                diff = 0  
            }  
            if (map[diff] !== undefined) {  
                res = Math.max(res, i - map[diff])  
            } else {  
                map[diff] = i  
            }  
        }  
    }  
}
```

```

    }

    }
    checkTwo('a', 'b')
    checkTwo('b', 'c')
    checkTwo('a', 'c')

    // Case 3
    let aCount = 0
    let bCount = 0
    let cCount = 0
    const map = {}
    map[0] = {}
    map[0][0] = -1
    for (let i = 0; i < s.length; i++) {
        if (s[i] === 'a') {
            aCount++
        } else if (s[i] === 'b') {
            bCount++
        } else {
            cCount++
        }
        const diffB = aCount - bCount
        const diffC = aCount - cCount
        if (map[diffB] && map[diffB][diffC] !== undefined) {
            res = Math.max(res, i - map[diffB][diffC])
        }
        map[diffB] ??= {}
        map[diffB][diffC] ??= i
    }
    return res
};

```

Java:

```

class Solution {
    public int longestBalanced(String s) {
        char[] c = s.toCharArray();
        int n = c.length;

        int cur_a = 0, cur_b = 0, cur_c = 0;
        int max_a = 0, max_b = 0, max_c = 0;

        for (int i = 0; i < n; i++) {

```

```

    if (c[i] == 'a') {
        cur_a = (i > 0 && c[i-1] == 'a') ? cur_a + 1 : 1;
        max_a = Math.max(max_a, cur_a);
    } else if (c[i] == 'b') {
        cur_b = (i > 0 && c[i-1] == 'b') ? cur_b + 1 : 1;
        max_b = Math.max(max_b, cur_b);
    } else {
        cur_c = (i > 0 && c[i-1] == 'c') ? cur_c + 1 : 1;
        max_c = Math.max(max_c, cur_c);
    }
}

int res = Math.max(Math.max(max_a, max_b), max_c);

res = Math.max(res, find2(c, 'a', 'b'));
res = Math.max(res, find2(c, 'a', 'c'));
res = Math.max(res, find2(c, 'b', 'c'));

res = Math.max(res, find3(c));

return res;
}

```

```

private int find2(char[] c, char x, char y) {
    int n = c.length, max_len = 0;
    int[] first = new int[2 * n + 1];
    Arrays.fill(first, -2);

    int clear_idx = -1, diff = n;
    first[diff] = -1;

    for (int i = 0; i < n; i++) {
        if (c[i] != x && c[i] != y) {
            clear_idx = i;
            diff = n;
            first[diff] = clear_idx;
        } else {
            if (c[i] == x) diff++;
            else diff--;

            if (first[diff] < clear_idx) {
                first[diff] = i;
            } else {
                max_len = Math.max(max_len, i - first[diff]);
            }
        }
    }
}

```

```

    }
    }
}

return max_len;
}

private int find3(char[] c) {
    long state = Long.MAX_VALUE / 2;
    Map<Long, Integer> first = new HashMap<>();
    first.put(state, -1);

    int max_len = 0;

    for (int i = 0; i < c.length; i++) {
        if (c[i] == 'a') state += 1_000_001;
        else if (c[i] == 'b') state -= 1_000_000;
        else state--;

        if (first.containsKey(state)) {
            max_len = Math.max(max_len, i - first.get(state));
        } else {
            first.put(state, i);
        }
    }

    return max_len;
}
}

```